

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Бэк-энд разработка

Отчет

Лабораторная работа №3  
Контейнеризация написанного приложения средствами  
docker

Выполнил:  
Христофоров Владислав

Группа  
К3340

Поток  
БР 1.2

Проверил:  
Добряков Д. И.

Санкт-Петербург

2026 г.

## Задача

Контейнеризировать микросервисное приложение. Реализовать файл Dockerfile, написать общий оркестрационный файл docker-compose.yml и настроить сетевое взаимодействие между всеми компонентами кластера (базы данных, брокер сообщений, микросервисы).

## Ход работы

### 1 Написание Dockerfile

Для сборки образов микросервисов разработан унифицированный конфигурационный файл Dockerfile, размещенный в корне директории services. В качестве базового образа выбран легковесный node:20-alpine. Процесс сборки разделен на логические слои:

- установка рабочей директории (WORKDIR /usr/src/app),
- копирование файлов package.json и установка зависимостей (npm install),
- копирование исходного кода и запуск компилятора TypeScript (npm run build),
- определение стартовой команды (CMD ["npm", "start"]), которая запускает скомпилированный JavaScript-код. Дополнительно создан файл .dockerignore для исключения директорий node\_modules и dist из контекста сборки.

### 2 Разработка единого docker-compose.yml

Разработан глобальный манифест docker-compose.yml, описывающий архитектуру системы как единое целое. В конфигурацию включены 10 независимых контейнеров:

- 4 контейнера баз данных (postgres:16-alpine), изолированных друг от друга (identity\_db, recipe\_db, social\_db, media\_db),
- 1 контейнер брокера сообщений (rabbitmq:3.8),
- 5 контейнеров Node.js микросервисов, собираемых «на лету» из написанного Dockerfile директивой build: context: ... .

### **3 Настройка сетевого взаимодействия и Environment Variables**

Все контейнеры объединены в единую виртуальную подсеть `culinary_network` с драйвером `bridge`. Служба DNS Docker автоматически разрешает имена контейнеров в IP-адреса. Таким образом, в переменные окружения (`environment`) микросервисов были переданы внутренние адреса.

Для управления порядком запуска применена директива `depends_on`, гарантирующая, что микросервисы начнут запуск только после инициализации БД и брокера. Для Media Service настроен `volume` (проброс тома) для постоянного хранения загруженных файлов на хост-машине. Во внешнюю сеть проброшен только порт 8000 от контейнера API Gateway и порты баз данных/брокера для администрирования.

#### **Вывод**

В ходе выполнения лабораторной работы микросервисное приложение было успешно контейнеризировано. Написание `Dockerfile` и конфигурация `docker-compose` позволили полностью автоматизировать процесс развертывания инфраструктуры одной командой `docker-compose up`. Настроенная сетевая изоляция и переменные окружения обеспечивают безопасное взаимодействие сервисов.